

**Nota: O exame deve ser respondido em 3 folhas de prova separadas:**

- 1 Folha para responder às três perguntas Teóricas
- 1 Folha para responder à 1ª pergunta da Prática
- 1 Folha para responder à 2ª pergunta da Prática

**Duração Total Exame (T + P) : 2h:00m + 30m tolerância**

**6 de Julho de 2023**

### Parte Teórica

- 1** Considere uma máquina com um espaço de endereçamento de memória RAM de 32 bits de bus de endereços, uma cache de 64 Kbytes de capacidade de dados, mapeamento directo e blocos de 1 byte.

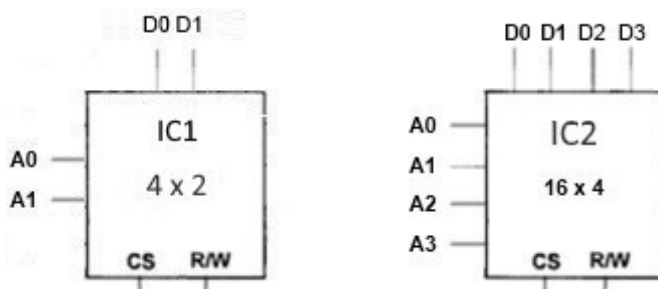
a) Quantos bits são necessários para a *tag*? Justifique.

**(1 Val.)**

b) Qual a capacidade total desta cache, contando com os dados, bits da *tag* e os *valid bits*?

**(2 Val.)**

- 2** Considere os circuitos integrados de memória RAM da figura, onde  $A_3, A_2, A_1, A_0$  representam linhas de endereço,  $D_3, D_2, D_1, D_0$  representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Selection*.



Faça um esboço, associando um número mínimo de circuitos integrados do tipo IC1, de forma a obter uma memória RAM com a capacidade e características do circuito IC2, identificando claramente as linhas de dados, de endereços, bem como as linhas R/W e CS da nova memória IC2.

**(2 Val)**

- 3** A gravação perpendicular e a gravação longitudinal utilizam formas distintas de representação da informação nos pratos dos discos rígidos. Descreva resumidamente as diferenças substanciais entre estas duas formas de gravação, fazendo referência às vantagens e inconvenientes de cada uma delas. Justifique a sua resposta.

**(2 Val)**

## Parte Prática

1. Considere um vetor **APOSTA** composto pelos 5 números e 2 estrelas do Euromilhões e um vetor **PPREMIO** igualmente composto por 5 números (valores compreendidos entre 1 e 50) e 2 estrelas (valores compreendidos entre 1 e 12).

Pode assumir que os dois vetores terão o número de elementos exato (5 números + 2 estrelas) e que os mesmos são valores naturais únicos. No entanto, o vetor **APOSTA** pode ter valores fora dos intervalos permitidos pelo concurso e tanto os números como as estrelas podem não estar ordenados de forma crescente.

Faça um programa que, dados os dois vetores **APOSTA** e **PPREMIO**, guarde as quantidades de números certos e de estrelas certas da **APOSTA** feita e ainda os valores inválidos nesse mesmo vetor **APOSTA**.

**(2,5 Val.)**

|                                                                                                                                         |                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exemplo 1:</b><br>PPREMIO = 1; 20; 23; 47; 49; 6; 7<br>APOSTA = 23; 45; 3; 40; 12; 4; 3<br>NUMEROS= 1<br>ESTRELAS= 0<br>INVALIDOS= 0 | <b>Exemplo 2:</b><br>PPREMIO = 1; 20; 23; 47; 49; 6; 7<br>APOSTA = 1; 47; 257; 99; 3; 0; 6<br>NUMEROS = 2<br>ESTRELAS = 1<br>INVALIDOS = 3 |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|

2. Construa um procedimento em *Assembly* que permita mostrar uma *string* em determinada posição do monitor, com uma direção/sentido específica. A *string* é terminada com um zero e a posição é especificada através da linha e coluna. Assuma que existem quatro direções/sentidos de escrita, especificados através de um valor numérico: 0 – horizontal, para a direita; 1 – horizontal para a esquerda; 2 – vertical, para cima; e 3 – vertical, para baixo.

Este procedimento **deve utilizar o acesso direto à memória de vídeo para realizar o output para o monitor (não é permitida a utilização de interrupções para aceder à memória de vídeo)**. Como parâmetros de entrada, deve receber: i) o endereço de memória da *string*, através do registo BX; ii) a posição no monitor (linha e coluna) onde escrever, através dos registos AL e AH, respetivamente; e iii) a direção/sentido de escrita, através do registo DL.

Veja o exemplo de utilização do procedimento apresentado em baixo (o resultado encontra-se a negrito e as direções/sentidos alternativos a cinza).

**(2,5 Val.)**

```
texto db 'ISEC', 0
```

```
lea BX, texto
mov AL, 5
mov AH, 10
mov DL, 2
call mostra
```

|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10       | 11 | 12 | 13 | 14 | 15 | ... |
|----|---|---|---|---|---|---|---|---|---|---|----------|----|----|----|----|----|-----|
| 0  |   |   |   |   |   |   |   |   |   |   |          |    |    |    |    |    |     |
| 1  |   |   |   |   |   |   |   |   |   |   |          |    |    |    |    |    |     |
| 2  |   |   |   |   |   |   |   |   |   |   | <b>C</b> |    |    |    |    |    |     |
| 3  |   |   |   |   |   |   |   |   |   |   | <b>E</b> |    |    |    |    |    |     |
| 4  |   |   |   |   |   |   |   |   |   |   | <b>S</b> |    |    |    |    |    |     |
| 5  |   |   |   |   |   |   |   |   | C | E | <b>S</b> | I  | S  | E  | C  |    |     |
| 6  |   |   |   |   |   |   |   |   |   |   | S        |    |    |    |    |    |     |
| 7  |   |   |   |   |   |   |   |   |   |   | E        |    |    |    |    |    |     |
| 8  |   |   |   |   |   |   |   |   |   |   | C        |    |    |    |    |    |     |
| 9  |   |   |   |   |   |   |   |   |   |   |          |    |    |    |    |    |     |
| 10 |   |   |   |   |   |   |   |   |   |   |          |    |    |    |    |    |     |
|    |   |   |   |   |   |   |   |   |   |   |          |    |    |    |    |    |     |

```
.8086
.model small
.stack 2048
DATA_HERE SEGMENT
...
DATA_HERE ENDS
CODE_HERE SEGMENT
    ASSUME CS:CODE_HERE, DS:DATA_HERE
START:mov ax, DATA_HERE
    mov ds, ax
    mov bx, 0b800h
    mov es, bx
    ...
    mov ah, 4ch
    int 21h
CODE_HERE ENDS
END START
```

BOA SORTE! ☺

T1

# Mapeamento Directo

T1 a)

Considere uma máquina com um espaço de endereçamento de 32 *bits*, uma cache de 64 Kbytes, mapeamento directo e blocos de 1 byte. Quantos *bits* são necessários para a tag?

A cache tem 64 K linhas, ou seja,  $2^6 \cdot 2^{10} = 2^{16}$ , logo o índice são 16 *bits*. A tag será de  $32 - 16 = 16$  *bits*.

T1 b)

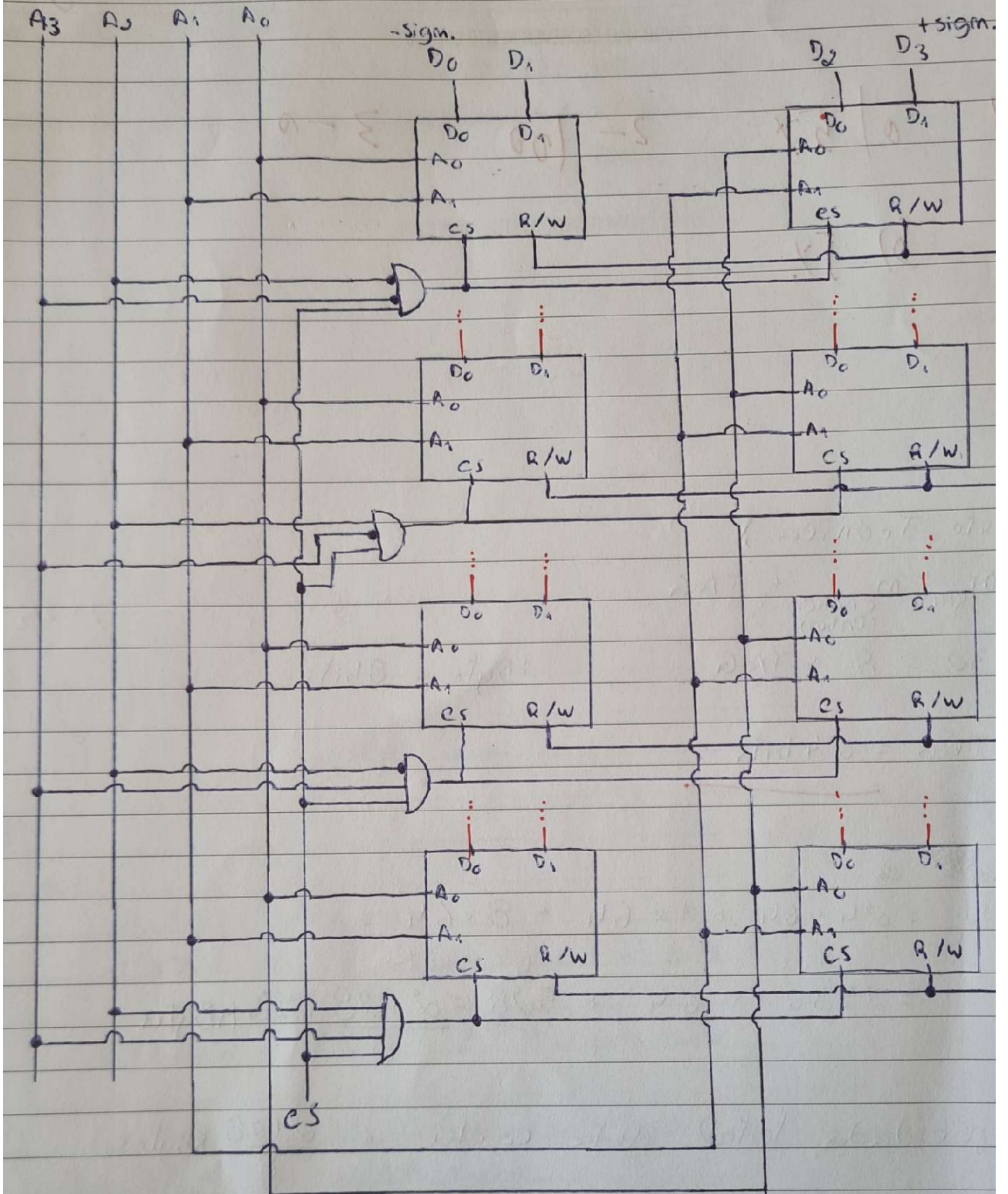
Qual a capacidade total desta cache, contando com os bits da tag mais os valid bits?

Temos 64 Kbytes de dados.

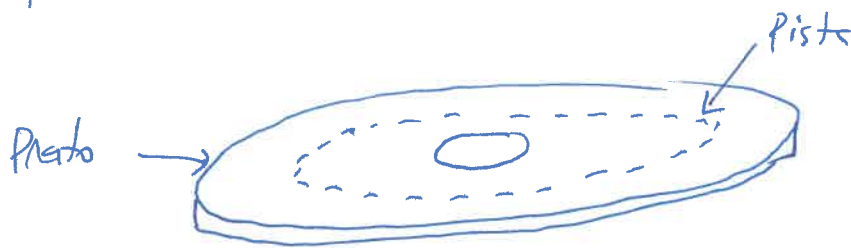
Cada linha tem 2 bytes de tag, logo 128Kbytes.

Cada linha tem um valid bit logo 64 Kbits = 8 Kbytes

Capacidade total =  $64 + 128 + 8 = 200$  KBytes

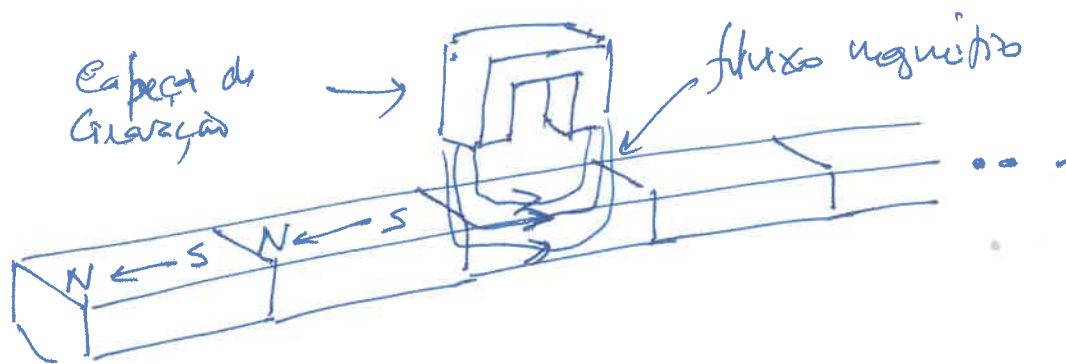


3- A gravação nos discos Rígidos é feita pelo processo de magnetização produzindo pequenas magnetizações ao longo de pistas circulares na superfície dos pratos que constitui o disco Rígido

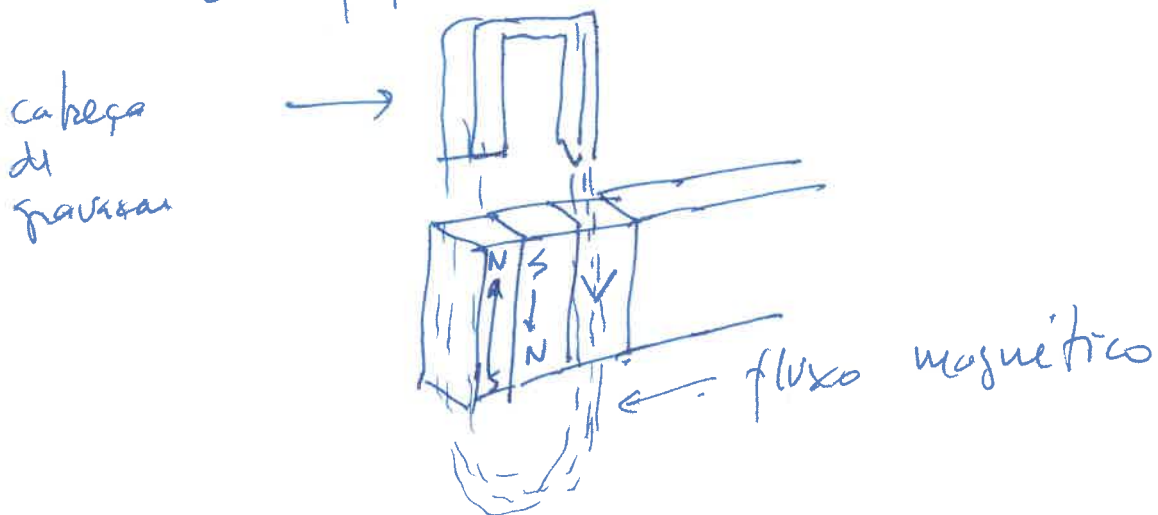


A magnetização longitudinal é a tecnologia mais antiga que sempre foi utilizada nos discos rígidos bem como também em fitas magnéticas

Vista em detalhe as sucessivas magnetizações produzidas por um fluxo magnético que é gerado pela cabeça de gravação vai produzindo pequenas magnetizações orientadas de acordo com o sentido do fluxo magnético



Na magnetização longitudinal representa a fita acima o fluxo magnético é essencialmente horizontal na zona da pista produzindo magnetização orientada horizontalmente na superfície de disco com orientações paralelas à superfície do ~~disco~~ prato.



Na magnetização perpendicular a cabeça de gravação tem uma forma distinta que produz a produção de um fluxo magnético perpendicular na zona magnetizada verticalmente.

Fazendo a comparação entre as duas formas de gravação facilmente se verifica que a gravação perpendicular permite uma densidade de gravação muito superior do que a magnetização longitudinal. pois permite muito mais magnetizações no mesmo espaço dimensional.

Portanto podemos dizer que a gravação perpendicular permite fazer gravações de mais bits no mesmo espaço físico. e consequentemente discos com muito mais capacidade de gravação.

P1

```

1 ;NOTA: dada a possibilidade dada durante o exame, o programa seguinte foi
desenvolvido para elementos com limite de 255. Para o caso do enunciado original, o
vector APOSTA deve ser definido como tipo WORD, os incrementos do SI devem ser feitos
de 2 em 2 e é necessário ao ler os elementos do PPREMIO que os mesmos sejam
guardados em registos de 16 bits (com a parte mais significativa a 0) de forma a que
se possa comparar com os elementos do tipo WORD do vector APOSTA

2
3 .8086
4 .MODEL SMALL
5 .STACK 1024
6 DADOS SEGMENT PARA 'DATA'
7     NUMEROS DB 0
8     ESTRELAS DB 0
9     INVALIDOS DB 0
10    PPREMIO DB 1, 20, 23, 47, 49, 6, 7
11    APOSTA DB 23, 45, 3, 40, 12, 4, 3
12    ;PPREMIO DB 1, 20, 23, 47, 49, 6, 7
13    ;APOSTA DB 55, 13, 254, 47, 3, 0, 6
14 DADOS ENDS
15 CODIGO SEGMENT PARA 'CODE'
16     ASSUME CS:CODIGO, DS:DADOS
17     main:
18         MOV AX, DADOS
19         MOV DS, AX ;inicialização DS
20
21         XOR SI,SI ;índice global para aceder aos elementos do vector APOSTA e para
saber com que tipo de elemento se está a trabalhar. NOTA: em caso do programa
necessitar de alterações a este índice, o mesmo deveria ser passado pela
pilha para uso dentro dos procedimentos
22
23         MOV CX, 7 ;Ciclo para percorrer os 7 elementos do vector APOSTA
24     cicloPrincipal:
25         PUSH CX ;garantir correcto funcionamento do ciclo principal
26         CALL prcValidarElemento
27         POP CX
28         INC SI
29         LOOP cicloPrincipal
30
31     fimPrograma:
32         MOV ah, 4ch
33         INT 21h
34
35     ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
36     ;procedimento para validar limites de um elemento conforme a sua posição
37     ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
38     prcValidarElemento PROC
39         XOR AX, AX
40         MOV AL, APOSTA[SI] ;guardar em AL valor do elemento actual do vector aposta
(apenas um BYTE) para validações e para passar como parâmetro para o
procedimento seguinte
41
42         ;testar limite inferior (quer seja número ou estrela)
43         CMP AX, 1
44         JB somaInvalido
45         ;testar limite superior (conforme seja número ou estrela)
46         CMP SI, 4
47         JBE validaNumero ;elementos que são números [0-4]
48         JMP validaEstrela ;elementos que são estrelas [5,6]
49     validaNumero:
50         CMP AX, 50
51         JA somaInvalido
52         JMP elementoValido
53     validaEstrela:
54         CMP AX, 12
55         JA somaInvalido
56
57     elementoValido:
58         CALL prcTestarPremio
59
60         RET
61     somaInvalido:
62         INC INVALIDOS
63     RET

```

```

64     prcValidarResultado ENDP
65
66     ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
67     ;procedimento para verificar se elemento em teste faz parte do vector PPREMIO
68     ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
69     prcTestarPremio PROC
70         ;inicializações para percorrer elementos do vector PPREMIO
71         CMP SI, 4
72         JA testaEstrela ;elementos que são estrelas [5,6]
73         XOR DI,DI
74         MOV CX, 5
75         JMP cicloPremio
76
77     testaEstrela:
78         ;elementos que são números [0-4]
79         MOV DI, 5
80         MOV CX, 2
81
82     cicloPremio:
83         MOV AL, PPREMIO[DI]
84         CMP APOSTA[SI], AL
85         JNE testarResultado
86         CALL prcIncrementaCertos
87         RET
88
89     testarResultado:
90         INC DI
91         LOOP cicloPremio
92     RET
93     prcTestarPremio ENDP
94
95     ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
96     ;procedimento para incrementar contadores de elementos certos da APOSTA
97     ;conforme o seu tipo
98     ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
99     prcIncrementaCertos PROC
100         CMP SI, 4
101         JBE incrementaNúmero ;elementos que são números [0-4]
102         INC ESTRELAS ;elementos que são estrelas [5,6]
103         JMP saiIncrementa
104
105     incrementaNúmero:
106         INC NUMEROS
107     saiIncrementa:
108     RET
109     prcIncrementaCertos ENDP
110
111
112 CODIGO ENDS
113 END main

```

```

1
2
3
4 ; EXAME (P2) - Possivel Solução (utiliza saltos condicionais)
5
6 ; cabecalho do procedimento
7 mostra proc
8 ; salvaguarda o contexto
9 pushf
10 push AX
11 push BX
12 push CX
13 push DX
14 push DI
15 push ES
16
17 ; segmento da memoria de video
18 mov CX, 0B800h
19 mov ES, CX
20
21 ; posicao de escrita no monitor
22 mov CX, AX
23 mov DH, 160 ; linha (AL) x 160
24 mul DH
25 mov DI, AX
26 mov AL, CH ; coluna (CH) x 2
27 xor AH, AH
28 add AX, AX
29 add DI, AX
30
31 ; determina salto relativo
32 cmp DL, 0
33 jne cmp1
34 mov CX, 2
35 cmp1:
36 cmp DL, 1
37 jne cmp2
38 mov CX, -2
39 cmp2:
40 cmp DL, 2
41 jne cmp3
42 mov CX, -160
43 cmp3:
44 cmp DL, 3
45 jne cmp0
46 mov CX, 160
47 cmp0:
48
49 ; escreve a palavra no monitor
50 mov AH, 01010011b
51 ciclo:
52 mov AL, [BX]
53 cmp AL, 0
54 je fim
55 mov ES:[DI], AX
56 add DI, CX
57 inc BX
58 jmp ciclo
59
60 fim:
61 ; recupera o contexto
62 pop ES
63 pop DI
64 pop DX
65 pop CX
66 pop BX
67 pop AX
68 popf
69
70 ; return
71 ret
72 mostra endp

```